`constexpr` placement new

Document #:	P2747R2
Date:	2024-03-19
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

1 Revision History
2 Introduction
3 Wording
    3.1 Feature-Test Macro
4 References

§ 1 Revision History

R0 [P2747R0] of this paper proposed three related features:

1. Allowing casts from *cv* `void*` to *cv* `T*` during constant evaluation
2. Allowing placement new during constant evaluation
3. Better handling an array of uninitialized objects

Since then, [P2738R1] was adopted in Varna, which resolves problem #1. Separately, #3 is kind of a separate issue and there are ongoing conversations about how to handle this in order to make `inplace_vector` [P0843R9] actually during during constant evaluation for all types. So this paper refocuses to just solve problem #2 and has been renamed accordingly.

Since [P2747R1], included support for arrays (yet another benefit over `std::construct_at`).

§ 2 Introduction

Consider this implementation of `std::uninitialized_copy`, partially adjusted from cppreference:

```
template <input_iterator I, sentinel_for<I> S, nothrow_forward_iterator I2>
constexpr auto uninitialized_copy(I first, S last, I2 d_first) -> I2 {
    using T = iter_value_t<I2>;
    I2 current = d_first;
    try {
        for (; first != last; ++first, (void)++current) {
            ::new (std::addressof(*current)) T(*first);
```

```
        }
    } catch (...) {
        std::destroy(d_first, current);
        throw;
    }
}
```

This fails during constant evaluation today because placement new takes a `void*`. But it takes a `void*` that points to a `T` - we know that by construction. It's just that we happen to lose that information along the way.

Moreover, that's not *actually* how `uninitialized_copy` is specified, we actually do this:

```
- ::new (std::addressof(*current)) T(*first);
+ ::new (voidify(*current)) T(*first);
```

where:

```
template<class T>
  constexpr void* voidify(T& obj) noexcept {
    return const_cast<void*>(static_cast<const volatile void*>
        (std::addressof(obj)));
  }
```

Which exists to avoid users having written a global placement new that takes a `T*`.

The workaround, introduced by [P0784R7], is a new library function:

```
template<class T, class... Args>
constexpr T* construct_at( T* p, Args&&... args );
```

This is a magic library function that is specified to do the same *voidify* dance, but which the language simply recognizes as an allowed thing to do. `std::construct_at` is explicitly allowed in [expr.const]/6:

> 6 […] Similarly, the evaluation of a call to `std::construct_at` or `std::ranges::construct_at` ([specialized.construct]) does not disqualify `E` from being a core constant expression unless the first argument, of type `T*`, does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of `E`, or the evaluation of the underlying constructor call disqualifies `E` from being a core constant expression.

It's good that we actually have a solution - we can make `uninitialized_copy` usable during constant evaluation simply by using `std::construct_at`. There's even a paper to do so ([P2283R2]). But that paper also had hinted at a larger problem: `std::construct_at` is an *extremely* limited tool as compared to placement new.

Consider the different kinds of initialization we have in C++:

kind	placement new	`std::construct_at`
value initialization	`new (p) T(args...)`	`std::construct_at(p, args...)`

kind	placement new	`std::construct_at`
default initialization	`new (p) T`	Not currently possible. [P2283R1] proposed `std::default_construct_at`
list initialization	`new (p) T{a, b}`	Not currently possible, could be a new function?
designated initialization	`new (p) T{.a=a, .b=b}`	Not possible to even write a function

That's already not a great outlook for `std::construct_at`, but for use-cases like `uninitialized_copy`, we have to also consider the case of guaranteed copy elision:

```
auto get_object() -> T;

void construct_into(T* p) {
    // this definitely moves a T
    std::construct_at(p, get_object());

    // this definitely does not move a T
    :::new (p) T(get_object());

    // this now also definitely does not move a T, but it isn't practical
    // and you also have to deal with delightful edge cases - like what if
    // T is actually constructible from defer?
    struct defer {
        constexpr operator T() const { return get_object(); }
    };
    std::construct_at(p, defer{});
}
```

Placement new is only unsafe because the language allows you to do practically anything - want to placement new a `std::string` into a `double*`? Sure, why not. But during constant evaluation we already have a way of limiting operations to those that make sense - we can require that the pointer we're constructing into actually is a `T*`. The fact that we have to go through a `void*` to get there doesn't make it unsafe.

Now that we have support for `static_cast<T*>(static_cast<void*>(p))`, we can adopt the same rules to make placement new work.

Additionally, `std::construct_at` does not support arrays (see also [LWG3436]), but we can make it work with placement without much issue - another reason the language is simply better:

```
int* p = std::allocator<int>{}.allocate(3);

new (p) int[]{1, 2, 3};       // ok
new (p + 1) int[]{2, 3};      // error (in this paper)
new (p) int[]{1, 2, 3, 4, 5}; // error
```

Note that the smaller array case could probably be made to work, but I'm not aware of a strong reason to want it - so it seems like a good start to allow the correct-size array case and reject the wrong-size array case for both directions of wrong size.

§ 3 Wording

Today, we have an exception for `std::construct_at` and `std::ranges::construct_at` to avoid evaluating the placement new that they do internally. But once we allow placement new, we no longer need an exception for those cases - we simply need to move the lifetime requirement from the exception into the general rule for placement new.

Clarify that implicit object creation does not happen during constant evaluation in 6.7.2 [intro.object]/14:

14 ~~An~~ Except during constant evaluation, an operation that begins the lifetime of an array of `unsigned char` or `std::byte` implicitly creates objects within the region of storage occupied by the array.

[*Note 5*: The array object provides storage for these objects. — *end note*]

~~Any~~ Except during constant evaluation, any implicit or explicit invocation of a function named `operator new` or `operator new[]` implicitly creates objects in the returned region of storage and returns a pointer to a suitable created object.

[*Note 6*: Some functions in the C++ standard library implicitly create objects ([obj.lifetime], [c.malloc], [mem.res.public], [bit.cast], [cstring.syn]). — *end note*]

Change 7.6.2.8 [expr.new]/15:

15 During an evaluation of a constant expression, a call to ~~an~~ a replaceable allocation function is always omitted ([expr.const]).

~~[*Note 1:* Only new-expressions that would otherwise result in a call to a replaceable global allocation function can be evaluated in constant expressions ([expr.const]). — *end note*]~~

Change 7.7 [expr.const]/5.18 (paragraph 14 here for context was the [P2738R1] fix to allow converting from `void*` to `T*` during constant evaluation, as adjusted by [CWG2755]):

(5.14) — a conversion from a prvalue `P` of type "pointer to cv `void`" to a "*cv1* pointer to `T`", where `T` is not *cv2* `void`, unless `P` points to an object whose type is similar to `T`;

(5.15) — …

(5.16) — …

(5.17) — …

(5.18) — a *new-expression* (7.6.2.8 [expr.new]), unless either

(5.18.1) — the selected allocation function is a replaceable global allocation function ([new.delete.single], [new.delete.array]) and the allocated storage is deallocated within the evaluation of `E`, or

(5.18.2) — the selected allocation function is a non-allocating form ([new.delete.placement]) with an allocated type `T`, where

(5.18.2.1) — the placement argument to the *new-expression* points to an object that is pointer-interconvertible with an object of type `T` or, if `T` is an array type, with the first element of an object of type `T`, and

(5.18.2.2) — the placement argument points to storage whose duration began within the evaluation of `E`;

Remove the special case for `construct_at` in 7.7 [expr.const]/6:

6 — For the purposes of determining whether an expression `E` is a core constant expression, the evaluation of the body of a member function of `std::allocator<T>` as defined in [allocator.members], where T is a literal type, is ignored. ~~Similarly, the evaluation of the body of `std::construct_at` or `std::ranges::construct_at` is considered to include only the initialization of the `T` object if the first argument (of type `T*`) points to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of `E`.~~

Change 17.6.2 [new.syn] to mark the placement new functions `constexpr`:

```
// all freestanding
namespace std {
  // [new.delete], storage allocation and deallocation
  [[nodiscard]] constexpr void* operator new  (std::size_t size, void* ptr)
        noexcept;
  [[nodiscard]] constexpr void* operator new[](std::size_t size, void* ptr)
        noexcept;
}
```

And likewise in 17.6.3.4 [new.delete.placement]:

```
[[nodiscard]] constexpr void* operator new(std::size_t size, void* ptr)
        noexcept;
```

2 *Returns*: `ptr`.

…

```
[[nodiscard]] constexpr void* operator new[](std::size_t size, void* ptr)
        noexcept;
```

5 *Returns*: `ptr`.

§ 3.1 Feature-Test Macro

Bump the value of `__cpp_constexpr` in 15.11 [cpp.predefined]:

```
- __cpp_constexpr 202306L
+ __cpp_constexpr 2024XXL
```

And add a new `__cpp_lib_constexpr_new` in 17.3.2 [version.syn]:

```
#define __cpp_lib_constexpr_new 2024XXL // freestanding, also in <new>
```

§ 4 References

[CWG2755] Jens Maurer. 2023-06-28. Incorrect wording applied by P2738R1.
https://wg21.link/cwg2755

[LWG3436] Jonathan Wakely. std::construct_at should support arrays.
https://wg21.link/lwg3436

[P0784R7] Daveed Vandevoorde, Peter Dimov,Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More constexpr containers.
https://wg21.link/p0784r7

[P0843R9] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2023-09-14. inplace_vector.
https://wg21.link/p0843r9

[P2283R1] Michael Schellenberger Costa. 2021-04-19. constexpr for specialized memory algorithms.
https://wg21.link/p2283r1

[P2283R2] Michael Schellenberger Costa. 2021-11-26. constexpr for specialized memory algorithms.
https://wg21.link/p2283r2

[P2738R1] Corentin Jabot, David Ledger. 2023-02-13. constexpr cast from void*: towards constexpr type-erasure.
https://wg21.link/p2738r1

[P2747R0] Barry Revzin. 2022-12-17. Limited support for constexpr void*.
https://wg21.link/p2747r0

[P2747R1] Barry Revzin. 2023-12-10. constexpr placement new.
https://wg21.link/p2747r1